

《计算机程序的构造和解释》期中考试（**参考答案和评分标准**）

(2022-2023 秋季学期, 授课教师: 冯新宇、李樾, 考试方式: 半开卷, 只允许携带一张 A4 纸资料)

学号: _____ 姓名: _____ 院系: _____ 得分: _____ (满分 100)

题目 得分	1	2	3	4	5	6	Extra

注意事项 (请仔细阅读):

1. 试题的难度不是线性递增的, **注意时间分配**。在这项测验中, 可能有些题目较难, 因此你不要在一道题上思考时间太久, 遇到不会答的题目可先跳过去, 如果有时间再去思考。否则, 你可能没有时间完成后面的题目。
2. 代码填空题的**正确答案不止一种**。代码填空题答案正确即可得分, 错误也不会倒扣分。每空**最多只能填一个**Python 语句或表达式, 可以直接使用的 Python 内置函数 (Python Built-in Functions) 包括: all, any, bool, dict, enumerate, filter, float, int, iter, len, list, map, max, min, pow, print, range, reversed, round, sorted, sum, zip。

1 Is it too difficult? (14pts)

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. Each expression has at least one line of output.

- If an error occurs, write **Error**, but include all output displayed before the error.
- To display a function value, write **Function**. To display an iterator value, write **Iterator**.
- If an expression would take forever to evaluate, write **Forever**.

The interactive interpreter displays the value of a successfully evaluated expression, unless it is None. Assume that you have first started python3 and executed the statements on the left.

```
1 def learner(Sclst):
2     def check(day):
3         get_score = iter(Sclst)
4         disappoint, oh_no = 0, False
5         for i in range(day):
6             score = next(get_score)
7             if score < 40: disappoint += 1
8             else: disappoint = 0
9             if disappoint >= 3:
10                 oh_no = True
11             if oh_no: print("Oh_no!")
12             elif score >= 90: print("Play!")
13             else: print("Work!")
14         return check
15
16 scoreLst1 = [47, 34, 21, 23, 91]
17 scoreLst2 = [38, 14, 63, 35, 95]
```

评分标准：3、2、3、2、4. 第 1、2、5 框每空 1 分，第 3 框第 1 空 2 分、第 2 空 1 分，第 4 框 2 分

Expression	Interactive Output
<code>pow(2, 11) - 25</code>	2023
<code>print(4, 5) + 1</code>	4 5 Error
<code>print(print(6), 9)</code>	6 None 9
<code>1 and [None] and 1/0</code> <code>not 0</code>	Error True
<code>digits = [4, 5, 6]</code> <code>[1, 2, 3] + digits * 2</code> <code>1 in digits</code>	[1, 2, 3, 4, 5, 6, 4, 5, 6] False
<code>list(range(4, 10, 2))</code>	[4, 6, 8]
Jack = learner(scoreLst1) Andy = learner(scoreLst2) Jack(2) Jack(5) Andy(1) Andy(5)	Work! Oh no! Work! Play!

2 Environment Diagram (14pts)

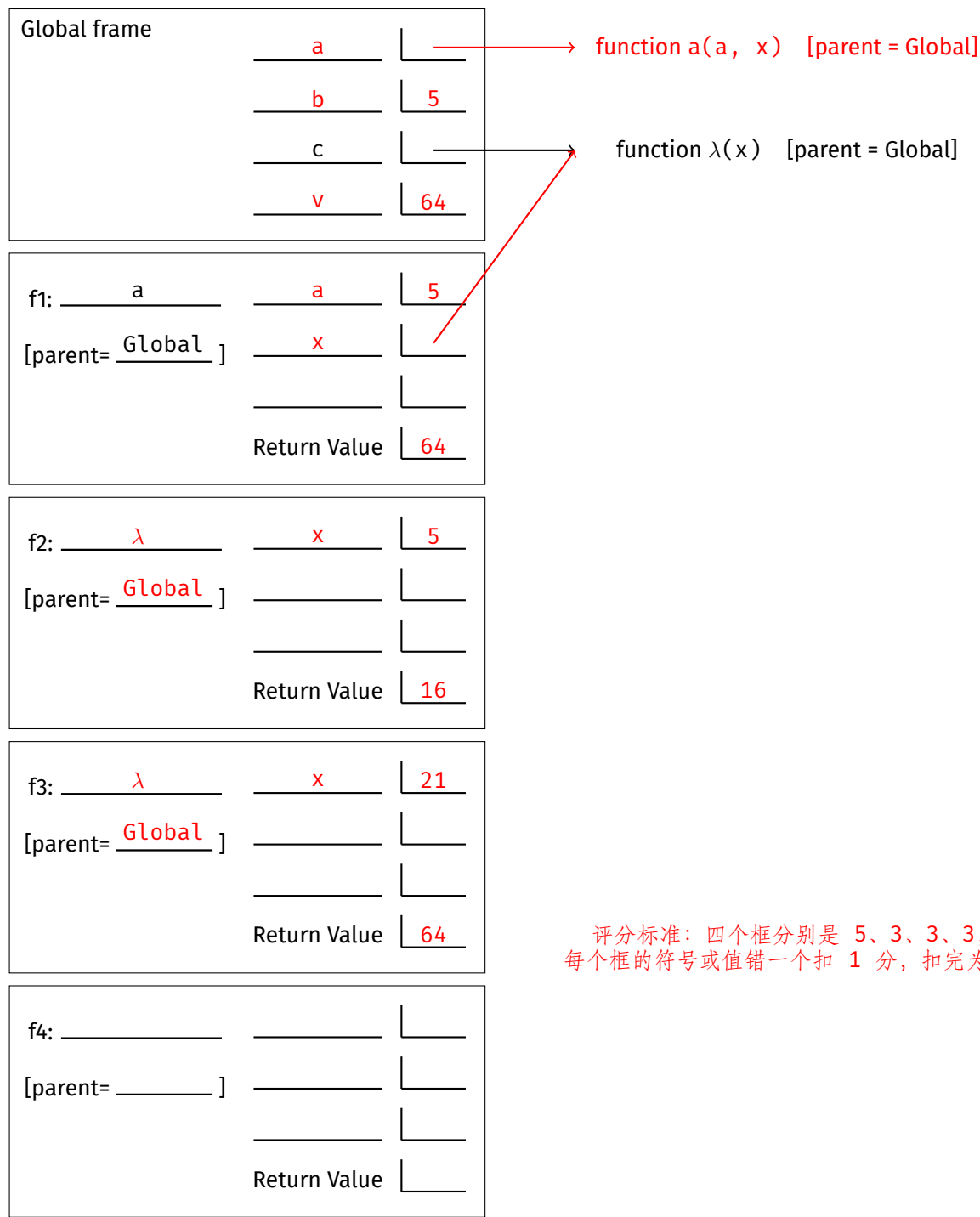
Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled.

You may not need to use all of the spaces or frames.

A complete answer will:

- Add missing names and parents to all local frames.
- Add missing values created or referenced.
- Show the return value for each local frame.

```
1 def a(a, x):
2     return c(a + x(a))
3 b = 5
4 c = lambda x: x * 3 + 1
5
6 v = a(b, c)
```



3 Lists (22pts)

Which expressions in the table can get the cake? Suppose we started python3 and executed the statements below. For each expression in the table, if evaluating it returns the string "cake", write **T**. Otherwise, write **F**. (5pts)

```
1 lst = [["trap"], [lambda: "cake"]]
2 lst[0].append(lst)
```

Expression	Your Answer
(lambda: "cake")()	T
(lambda x: "cake")()	F
lst[1][0]()	T (1分)
lst[0][1][1]	F (1分)
lst[0][1][1]()	F (1分)
(lst[0][1:]+[lst])[2][1][0]()	F (1分)
lst.pop()[1][1]()	F (1分)

Write expressions involving the list `l` that evaluate to the following values.
Each expression shall be a **single list comprehension** involving the list `l`. (4pts)

```
1 >>> l = [[1,9], [2,8], [3,7], [4,6], [5,5]] + [[0, 11]]
2 >>> [x for x, y in l if y < 8] # an example for you
3 [3, 4, 5]
4 >>> [[x, y] for x, y in l if x % 2 != 0 and y % 2 != 0] (2分)
5 [[1,9], [3,7], [5,5]]
6 >>> [y - x for x, y in l[:5]] (2分)
7 [8, 6, 4, 2, 0]
```

A **prefix sum** of a sequence of numbers is the sum of the first n elements for some positive length n .

Implement `prefix`, which takes a list of numbers `s` and returns a new list such that the n -th element of the new list is the prefix sum of the first n elements of `s`. The expected answer is not long and may use list slicing and ranges. (4pts)

```
1 def prefix(s):
2     """Return a list of all prefix sums of list s.
3     >>> prefix([1, 2, 3, 0, 4, 5])
4     [1, 3, 6, 6, 10, 15]
5     >>> prefix([2, 2, 2, 0, -5, 5])
6     [2, 4, 6, 6, 1, 6]
7     """
8     return [sum(s[:k+1]) (2分) for k in range(len(s)) (2分)]
```

Draw the final environment diagrams for the following code. Think twice before your write! (9pts)

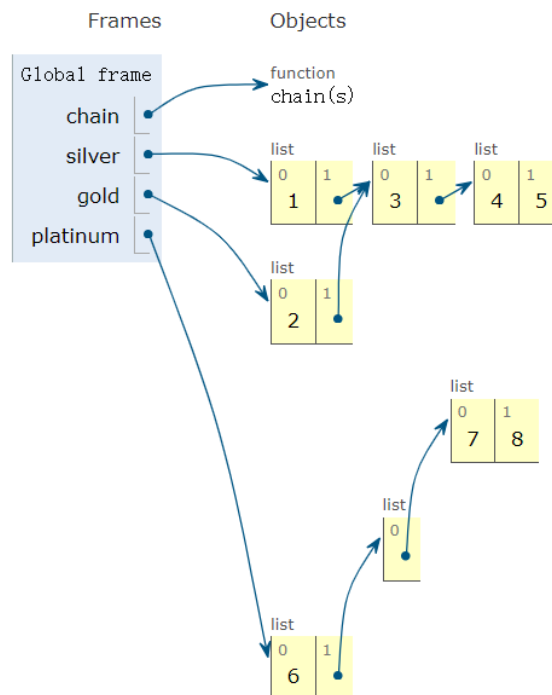
```

1 def chain(s):
2     return [s[0], s[1:]]
3 silver = [2, chain([3, 4, 5])]
4 gold = [silver[0], silver[1]]
5 silver[0] = 1
6 platinum = chain(chain([6, 7, 8]))

```

Global frame

chain	└
silver	└
gold	└
platinum	└



评分标准: chain 1 分, silver 3 分, gold 2 分, platinum 3 分。

4 Higher-Order Functions (12pts)

Implement `curry`. `curry` takes a binary function `f`, returns a new function `g` such that `g(a)(b)=f(a,b)`. (3pts)

```
1 def curry(f):
2     """ Returns a function g, such that g(a)(b) = f(a, b).
3     >>> curried_add = curry(lambda a, b: a + b)
4     >>> curried_add(3)(5)
5     8
6     """
7     return lambda a: lambda b: f(a, b) (3分)
```

Implement `flip`. `flip` takes a **curried** binary function `g`, returns a new function `h` that swaps `g`'s arguments. For example, if `g` is `(lambda a: lambda b: a + b)`, then `h` should be `(lambda b: lambda a: a + b)`. (3pts)

```
1 def flip(g):
2     """ Returns a function h, such that h(a)(b) = g(b)(a).
3     >>> g = curry(lambda a, b: a - b)
4     >>> g(2)(1), flip(g)(2)(1), flip(flip(g))(2)(1)
5     (1, -1, 1)
6     """
7     return lambda a: lambda b: g(b)(a) (3分)
```

Implement `fold`. `fold` takes a **curried** binary function `g` and a 3-tuple `t=(a,b,c)`. It transforms the 3-tuple to a value. For example, `fold(lambda x: lambda y: x + y, (1, 2, 3))` is `(1 + 2) + 3`

```
1 def fold(g, t):
2     """ Folds a 3-tuple to a value.
3     >>> fold(lambda x: lambda y: x + y, (1, 2, 3)) # (1 + 2) + 3
4     6
5     >>> fold(lambda x: lambda y: x - y, (2, 3, 4)) # (2 - 3) - 4
6     -5
7     """
8     (x, y, z) = t
9     return g(g(x)(y))(z) (3分)
```

Finally, suppose you have implemented `flip` and `fold` correctly, what would python display? Fill in the blanks. (3pts)

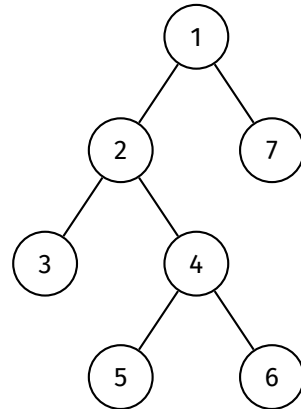
```
1 >>> flip(lambda x:x)(1)(lambda x: x + 1)
2 2 (1分)
3 >>> fold(flip(lambda x:x), (1, lambda x: x + 1, lambda x: x * 2))
4 4 (2分)
```

5 Trees (23pts)

This problem is based on the assumption that all labels in a tree are different positive numbers.

The following tree APIs are available for this problem:

- `tree(label, branches=[])`
- `label(tree)`
- `branches(tree)`
- `is_leaf(tree)`
- When printing a tree, you will get a string that, when executed, construct the same tree.



Define the tree in the figure using tree constructor. (3pts)

```
t1 =  
tree(1, [tree(2, [tree(3), tree(4, [tree(5),  
tree(6)]])], tree(7)) (3分)
```

评分标准: 自第1层 (2/7 节点所在层) 开始, 写对一层给1分。

Implement `count_leaves`, which takes in a tree and count the number of its leaves. (3pts)

```
1 def count_leaves(t):  
2     """ Returns the number of leaves in this tree.  
3     >>> count_leaves(t1) # 3, 7, 5, 6  
4     4  
5     """  
6     if is_leaf(t) (1分):  
7         return 1  
8     return sum([count_leaves(b) for b in branches(t)] (2分))
```

Implement `add_one`, which takes in a tree and returns a new tree with each node's label is the corresponding node in original tree's label plus one. (4pts)

```
1 def add_one(t):  
2     """ Returns a new tree, with 1 added to each node.  
3     >>> add_one(t1)  
4     tree(2, [tree(3, [tree(4), tree(5, [tree(6), tree(7)]])], tree(8))  
5     """  
6     return tree(label(t)+1 (1分), [add_one(b) for b in branches(t)] (3分))
```

Implement `count_leaves_at_depth`, which counts how many leaves are there in a tree with given depth. (5pts)

The depth of a node is defined as follows:

- If it is the root, its depth is 0.
- Otherwise, its depth is the depth of its parent plus one.

```
1 def count_leaves_at_depth(t, depth):  
2     """ Return the number of leaves at depth.  
3     >>> count_leaves_at_depth(t1, 1) # 7  
4     1  
5     """  
6     def helper(t, current_depth):
```

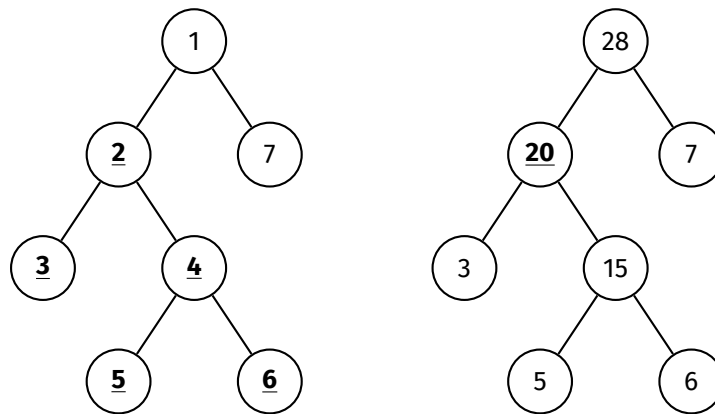
```

7         if current_depth==depth:
8             if is_leaf(t) (1分) :
9                 return 1
10            return 0
11        leaves=[helper(b,current_depth+1) for b in branches(t)] (3分)
12        return sum(leaves)
13    return helper(t,0) (1分)

```

Implement `sum_sub_trees`, which takes in a tree `t` and returns a new tree of the same shape but with new labels. Each new label sums up the old label of the same node and all the old labels in the sub-trees of this node (by old label we mean the label in the old tree). (5pts)

For example, when calculating `sum_sub_tree` for `t1`, we would get this result. Here, $20 = 2 + 3 + 4 + 5 + 6$.



```

1 def sum_sub_trees(t):
2     """ Returns a new tree, with each node's label replaced by the sum of that sub tree.
3     >>> sum_sub_trees(t1)
4     tree(28, [tree(20, [tree(3), tree(15, [tree(5), tree(6)])]), tree(7)])
5     """
6     added=[sum_sub_trees(i) (1分) for i in branches(t)]
7     return tree(
8         label(t)+sum([label(i) for i in added]) (3分) ,
9         added (1分) )

```

Implement `find_node`, which takes a tree `t` and a label value `v`, finds the node in `t` with the same label, and returns the sub-tree of `t` whose root is this node. If no such node is found, `find_node` returns `None`. (3pts)

```

1 def find_node(t, v):
2     """ Returns the node with given label value.
3     >>> find_node(t1, 3)
4     tree(3)
5     """
6     if label(t) == v (1分) : return t
7     else:
8         findings = [find_node(b, v) (2分) for b in branches(t)]
9         for f in findings:
10             if f: return f
11        return None

```


6 Hofstadter Figure-Figure Sequences (15pts)

Hofstadter Figure-Figure Sequences R and S is defined by:

$$R(1) = 1 \quad (1)$$

$$S(1) = 2 \quad (2)$$

$$R(n) = R(n-1) + S(n-1) \quad (3)$$

with the sequence $S(n)$ defined as the smallest positive number which not occurs in $R(1), R(2), \dots, R(n)$ and $S(1), S(2), \dots, S(n-1)$.

Note:

- $R(n)$ and $S(n)$ are all defined on positive integers, you don't need to care about $R(0)$ or $S(0)$

You should write down the first 5 terms of the $R(n)$ and $S(n)$ (4pt)

$$R(1) = 1 \quad S(1) = 2 \quad (4)$$

$$R(2) = 3 \quad S(2) = 4 \quad (5)$$

$$R(3) = 7 \quad S(3) = 5 \quad (6)$$

$$R(4) = 12 \quad S(4) = 6 \quad (7)$$

$$R(5) = 18 \quad S(5) = 8 \quad (8)$$

(9)

To calculate $R(n)$ and $S(n)$ in python, we use 2 mutually recursion functions, $r(n)$ and $s(n)$. But before this, we first notice that $R(n)$ and $S(n)$ satisfy the following properties (please fill in the blank with $<$, $\leq$, $=$, $\geq$, or $>$): (2pt)

$$S(n) \geq 2 \quad (10)$$

$$R(n+1) - R(n) \geq 2 \quad (11)$$

With this property, it's easy to implements $r(n)$ and $s(n)$. (5pt)

```
1 def r(n):
2     if n == 1:
3         return 1
4     else:
5         return r(n - 1) + s(n - 1) (3分)
6
7 def in_r(val, limit):
8     """
9     returns if val exists in [ r(0), r(1), ... r(limit) ]
10    """
11    rs = [ r(i) for i in range(1, limit + 1) ]
12    return val in rs
13
14 def s(n):
15     if n == 1:
```

```

16         return 2
17     else:
18         vs = s(n - 1)
19         if in_r(vs + 1, n):
20             return vs + 2 (1分)
21         else:
22             return vs + 1 (1分)

```

But the above code is very slow. We can speed it up by caching the compute results in two arrays, rn and sn, and the hole program can be written as a loop. (3pt)

```

1 def rs(n):
2     """
3     returns the value of r(n) and s(n)
4     >>> rs(1)
5     (1, 2)
6     """
7     rn = [-1, 1]
8     sn = [-1, 2]
9     i = 1
10    while i < n (1分) :
11        rn.append(rn[i] + sn[i])
12        if sn[i] + 1 in rn:
13            sn.append(sn[i] + 2) (1分)
14        else:
15            sn.append(sn[i] + 1) (1分)
16        i += 1
17    return rn[n], sn[n]

```

Extra Problems

- Genshin Impact is a popular game.
- Complete the following python script, so that when your computer screen is pure white, start genshin impact. (1pts)

```
1 import subprocess
2 from PIL import ImageGrab
3 import time
4
5 def is_all_white(image):
6     # If r, g and b are all equal to or greater than this, we regard it as white
7     white_threshold = 200
8     # Only when this much of the screen is white do we regard it as pure white
9     pure_white_ratio = 0.9 # 90% of screen should be white
10
11     white_count=0
12     for y in range(0, image.height):
13         for x in range(0, image.width):
14             pixel = image.getpixel((x, y))
15             # pixel = [red, green, blue]
16             if all([ i >= white_threshold for i in pixel ]) (1分) :
17                 white_count+=1
18     return white_count/(image.width*image.height) >= pure_white_ratio
19
20 def start_genshinimpact():
21     # You may assume that this path is correct
22     subprocess.Popen(r"C:\Program Files\Genshin Impact\launcher.exe")
23
24 def main():
25     while True:
26         # Take a screenshot
27         screenshot = ImageGrab.grab()
28
29         if is_all_white(screenshot):
30             start_genshinimpact()
31
32         time.sleep(0.5)
33
34 if __name__ == "__main__":
35     # When the file is directly executed, rather than imported, this will be run
36     main()
```

- Tim thinks `is_all_white` is slow. For example, if you have a display of 3840x2160, the if clause (line 14) would be run **8294400 (1分)** times every 0.5 seconds.
- Speed up `is_all_white` under normal cases. Hint: normally the check would fail, because lots of pixels cannot be regarded as white. (1pts)

```
1 def is_all_white(image):
2     # If r, g and b are all equal to or greater than this, we regard it as white
3     white_threshold = 200
4     # Only when this much of the screen is white do we regard it as pure white
5     pure_white_ratio = 0.9 # 90% of screen should be white
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
```